COMPTIA PENTEST+ · **PT0-003** · DOMAIN 4.10

Scripting & Automation

Domain 4.10 is where the exam tests whether you can **read a script and know what it does**, pick the right automation tool for a target, and tell near-identical frameworks apart. This guide covers the four toolsets — PowerShell, Bash, Python, and Breach & Attack Simulation — and adds a **Script Lab**: token-by-token anatomy, spot-the-script drills, and fix-the-broken-script exercises. Tap any part of a command to learn what it does.

4 toolsets · 12 tools/concepts | Objective verb: Use / Analyze | Heavy on: reading code

Read Code | 1 · PowerShell | 2 · Bash | 3 · Python | 4 · BAS | Script Lab | Correlations | Strategy

READ How to Read a Script

The exam shows you code and asks "what does this do?" or "which line is wrong?" You don't need to write scripts from scratch — you need to **parse** them: identify the language, break a command into its parts, and trace data left-to-right. Start here, then use the same lens in every section below.

A Spot the language in one glance

Before reading meaning, pin the language — it narrows what every symbol can mean. These tells are almost always enough:

IF YOU SEE…	IT'S…
`Verb-Noun` cmdlets, `-Param`, `$var`, `-eq/-ne`	PowerShell — Windows/AD automation
`|` pipes into grep/awk/sed/cut, `2>/dev/null`, `$1`	Bash — Linux text-stream automation
`import`, `def`, `IP()/TCP()`, `.py` tools	Python — full programming (Impacket / Scapy)
`Invoke-AtomicTest`, ATT&CK IDs like `T1003.001`, YAML	BAS — attack-simulation frameworks

⚑ EXAM TELL

A `$` means different things per language: in Bash `$1` is a positional argument and `$VAR` a variable; in PowerShell `$true` / `$null` are automatic variables. Same symbol, different rules — language first, meaning second.

B The parts of a command

Almost every line breaks into the same handful of roles. Tap each colored part below to see what it does — this is the exact dissection you'll do on the exam.

⟲ ANATOMY — BASH · ONE-LINE RECON PIPELINE

```
nmap -p 80,443 "$1" 2>/dev/null | grep "open" | awk '{print $1}' | sort -u | tee web_targets.txt
```

1. **Command** — the tool being run. The first word of any segment.
2. **Flag + value** — `-p 80,443` restricts the scan to ports 80 and 443.
3. **Quoted argument** — `"$1"` is the first positional parameter (the target you passed in). Quotes stop word-splitting.
4. **Stream redirect** — `2>/dev/null` throws away stderr so error noise doesn't pollute the pipeline.
5. **Pipe** — sends the previous command's stdout into the next command's stdin. Read pipes left → right.
6. **Filter** — `grep "open"` keeps only lines containing "open".
7. **Field extract** — `awk '{print $1}'` prints the first whitespace-delimited column (the IP).
8. **Dedupe** — `sort -u` sorts and removes duplicates.
9. **tee** — writes results to the file *and* passes them on/prints them. Log + display at once.

⇄ READ IT AS A SENTENCE

"Scan ports 80/443 on the target, hide errors, keep open lines, take the IP column, dedupe, and save to a file while showing it." If you can narrate a pipeline like this, you can answer any "what does this script do?" question.

SEC 01 PowerShell Automation

PowerShell ships on every modern Windows box and reaches deep into the OS, Active Directory, the registry, and remote systems. Because it uses legitimate Windows infrastructure, PowerShell attacks blend into normal admin activity — **"living off the land."** Four toolsets matter, split by *target*: local host, AD, SQL Server.

1.1 PowerSploit

A modular **library of PowerShell post-exploitation scripts** — not one tool. Its signature trait is running code **in-memory** so nothing malicious touches disk, reducing forensic artifacts. Key modules: **PowerUp** (privilege escalation — checks unquoted service paths, weak service perms, writable registry keys, AlwaysInstallElevated); **Invoke-Mimikatz** (dumps credentials from LSASS memory); plus code execution, persistence, and exfiltration modules.

⚑ EXAM TELL

PowerSploit = **post-exploitation**, not initial access. Learn the module→job pairing: **PowerUp = privesc**, **Invoke-Mimikatz = credential dumping**. Modern EDR detects it — the exam tests what it *does*, not that it evades defenses.

EXAM FOCUS

- Operates in-memory (fileless) to avoid detection.
- PowerUp = privilege escalation; Invoke-Mimikatz = credential harvesting.
- Also surfaces under Domain 3.1 (vuln discovery) and 5.1 (persistence).

`post-exploitation` `in-memory execution` `PowerUp` `unquoted service paths` `credential dumping` `fileless` `living off the land`

1.2 PowerView

A PowerShell tool for **Active Directory enumeration and situational awareness**. Part of PowerSploit but used standalone. It maps out users, groups, computers, trusts, sessions, and permissions from the command line. Core cmdlets: `Get-NetUser`, `Get-NetGroupMember` (find "Domain Admins" members), `Get-NetComputer`, `Get-NetDomainTrust`, and the crown jewel `Find-DomainUserLocation` — **it finds where Domain Admins are currently logged in**. It also analyzes ACLs (`Find-InterestingDomainAcl`) to spot misconfigured permissions that enable privesc.

⇄ DISTINGUISH

PowerView = **AD user/group/session enumeration**. Its unique differentiator is *locating logged-in Domain Admins*. It's the command-line companion to **BloodHound**, which visualizes AD attack paths graphically (via the SharpHound collector).

EXAM FOCUS

- Primary purpose = AD enumeration; finds where Domain Admins are logged in.
- Trust enumeration supports cross-domain attacks.
- Choose PowerView (AD enum) vs. PowerUpSQL (SQL) vs. BloodHound (visualization).

`Active Directory` `Domain Admins` `session discovery` `trust relationships` `ACL analysis` `BloodHound` `lateral movement planning`

1.3 PowerUpSQL

A PowerShell toolkit for **attacking Microsoft SQL Server**. It discovers instances (even without a scan — `Get-SQLInstanceDomain` finds SQL Servers registered as **SPNs in Active Directory**), audits them (`Invoke-SQLAudit`), and escalates. The two capabilities to remember: **xp_cmdshell** — when enabled, runs OS commands as the SQL service account (often local admin/SYSTEM); and **linked servers** (`Get-SQLServerLinkCrawl`) — SQL-to-SQL trust chains that create a **lateral-movement path bypassing network controls**.

⚑ EXAM TELL

Scenario says "SQL Server" → PowerUpSQL. The two magic words are **xp_cmdshell** (SQL access → OS command execution) and **linked servers** (lateral movement between databases). A weak/default `sa` password under mixed-mode auth is the classic first step.

EXAM FOCUS

- SQL Servers are discoverable via AD SPNs, not just network scans.
- xp_cmdshell = SQL → OS command execution.
- Linked servers = lateral movement across DB infrastructure.

`SQL Server` `xp_cmdshell` `linked servers` `SPN enumeration` `sa account` `sysadmin` `mixed mode`

1.4 AD Search (native cmdlets)

Using PowerShell's **built-in ActiveDirectory module** to automate LDAP queries — `Get-ADUser`, `Get-ADGroup`, `Get-ADComputer`, `Get-ADTrust`. Because it uses Microsoft's own ADWS interface, it's **stealthier than third-party tools** like PowerView. The power is in `-Filter`: find stale accounts, `PasswordNeverExpires` accounts, and — most exam-relevant — accounts with SPNs set (**Kerberoasting targets**). Output pipes cleanly to `Export-CSV` for reporting.

⟲ ANATOMY — POWERSHELL · FIND KERBEROASTING TARGETS

```
Get-ADUser   -Filter   {ServicePrincipalName -ne "$null"}   -Properties   ServicePrincipalName
```

1. **Cmdlet (Verb-Noun)** — `Get-ADUser` queries user objects. The Verb-Noun shape is the PowerShell fingerprint.
2. **Parameter** — `-Filter` constrains which objects come back (a server-side LDAP filter).
3. **Filter expression** — "users whose ServicePrincipalName is not null," i.e. accounts that *have an SPN*.
4. **Parameter** — `-Properties` asks AD to also return the named attribute (not loaded by default).
5. **Property** — return the SPN itself. Accounts with SPNs are Kerberoastable — the whole point of this query.

⚑ EXAM TELL

"Enumerate accounts with SPNs" → AD Search feeding **Kerberoasting**. Native cmdlets are the stealthier choice; PowerView is offensive-specific; BloodHound visualizes. Match the phrasing to the tool.

EXAM FOCUS

- Automates LDAP queries via the native AD module (ADWS interface).
- SPN enumeration → Kerberoasting; filters find stale/privileged accounts.
- Produces structured output (Export-CSV) for reporting.

`LDAP` `Service Principal Names` `Kerberoasting` `Get-ADUser` `Get-ADGroupMember` `RSAT` `ADWS`

SEC 02 Bash Scripting

Bash is the default Linux/macOS shell and works on **text streams** — reading, filtering, transforming text.
Its superpower is chaining existing tools into pipelines. Two capabilities carry this domain: **I/O management** (where data flows) and **data manipulation** (how it's transformed).

2.1 Input / Output Management

Every process has three streams: **stdin (0)**, **stdout (1)**, **stderr (2)**. Controlling them is the foundation of automation.

BASH · REDIRECTION

```bash
nmap -sV host > scan.txt        # overwrite stdout to a file
nmap -sV host >> scan.txt       # APPEND instead of overwrite
nmap -sV host > out.txt 2>&1     # both stdout AND stderr to one file
nmap -sV host 2>/dev/null        # discard error noise
cat targets.txt | while read h; do nmap -sV "$h"; done   # feed a list
```

The pipe `|` connects one command's stdout to the next's stdin. `tee` writes to a file *and* keeps output flowing. `$1`, `$2` are positional arguments; heredocs (`<<EOF`) supply multi-line input; process substitution `<(cmd)` lets a command's output stand in for a file.

⚑ EXAM TELL

Two symbol pairs are tested constantly: `>` overwrites vs `>>` appends, and `2>&1` means "send stderr wherever stdout is *currently* going" — so **order matters**: it must come *after* the `> file` redirect, or stderr still goes to the terminal.

EXAM FOCUS

- `>` overwrite, `>>` append, `2>/dev/null` discards errors.
- `|` = stdout → stdin; `tee` = log + display simultaneously.
- `$1/$2` = positional parameters; trace what a script does by its operators.

stdin · stdout · stderr · redirection · pipe · append · /dev/null · tee · heredoc · positional parameters

2.2 Data Manipulation

Pentest tools spew text; these six turn it into intel. Learn each one's *one job* — that's what the exam asks.

BASH · THE CORE SIX

```bash
grep "open" scan.txt        # FILTER: keep matching lines (-v inverts, -i ignores case)
cut -d: -f1 /etc/passwd      # EXTRACT: field 1, colon-delimited
awk '{print $1}' scan.txt    # FIELD PROCESSING + math on columns
sed 's/http/https/g' urls    # FIND/REPLACE stream editing
sort ports.txt | uniq -c     # ORDER + COUNT (uniq needs SORTED input!)
grep "open" *.txt | awk '{print $1}' | sort -u   # a real pipeline
```

⇄ DISTINGUISH

grep filters, **cut** grabs fixed fields, **awk** processes fields (and does math), **sed** find-and-replaces, **sort | uniq** orders and dedupes. The classic trap: `uniq` only collapses *adjacent* duplicates, so it's almost always preceded by `sort`.

EXAM FOCUS

- grep (filter), cut (fields), awk (field math), sed (replace), sort/uniq (order+dedupe).
- `grep -v` inverts; `sort | uniq -c` counts occurrences.
- Read a pipeline left→right; state the output it produces.

grep · awk · sed · cut · sort · uniq · regex · field delimiter · text processing

SEC 03 Python Scripting

Where Bash chains tools and PowerShell runs Windows, Python gives **full programming** — sockets, packet crafting, protocols, crypto. Two libraries own this domain: **Impacket** (Windows network protocols) and **Scapy** (raw packets).

3.1 Impacket

A collection of Python classes/tools implementing **Windows network protocols** — SMB, MSRPC, NTLM, Kerberos, LDAP — so you can attack Windows **from a Linux box** like Kali. Memorize the flagship `.py` tools by job:

- **secretsdump.py** — credential dumping (SAM, LSA secrets, NTDS.dit from a DC).
- **psexec.py / wmiexec.py / smbexec.py / atexec.py** — remote execution. psexec drops a service; wmiexec runs via WMI (stealthier, no service).
- **GetUserSPNs.py** — Kerberoasting (request tickets for SPN accounts to crack offline).
- **GetNPUsers.py** — AS-REP Roasting (accounts with pre-auth disabled).
- **ntlmrelayx.py** — NTLM relay (pairs with Responder poisoning LLMNR/NBT-NS).
- **ticketer.py** — forge Golden/Silver Kerberos tickets.

> ⚑ EXAM TELL
>
> Windows protocols (SMB/Kerberos/NTLM/LDAP) → Impacket. Know the relay chain cold: **Responder poisons → ntlmrelayx.py relays → access gained**. And `GetUserSPNs.py` = Kerberoasting service accounts.

EXAM FOCUS

- Attacks Windows from non-Windows systems.
- secretsdump=dump, psexec=exec, ntlmrelayx=relay, GetUserSPNs=Kerberoast.
- Appears across Domains 4.2, 4.3, 5.2, and 4.10.

SMB · NTLM relay · Kerberoasting · AS-REP roasting · NTDS.dit · Golden Ticket · secretsdump · psexec · ntlmrelayx

3.2 Scapy

A Python library for **raw packet manipulation** — build packets field-by-field and put them on the wire. Its signature is **layer stacking with the `/` operator**: `IP()/TCP()`. It replaces bits of Nmap, hping, and more, and does things no single tool was built for: ARP spoofing, VLAN hopping (double-tagging), custom port scans, DNS spoofing, firewall/IDS testing, and protocol fuzzing (`fuzz()`).

⟲ ANATOMY — PYTHON · SCAPY SYN SCAN

```
ans, unans  =  sr ( IP(dst="192.168.1.1")  /  TCP(dport=[22,80,443], flags="S") ,  timeout=2 )
```

1. **Two return values** — `sr()` returns a *tuple*: answered and unanswered packets. This is why `sr` ≠ `sr1`.
2. **send-receive** — `sr()` sends packets and collects replies. `sr1()` returns only the first response (single packet).
3. **Layer 3** — the `IP()` layer sets the destination address.
4. **Stack operator** — `/` stacks layers, outer→inner. `IP()/TCP()` = a TCP segment inside an IP packet. Reversing it is a bug.
5. **Layer 4** — `TCP()` with a *list* of destination ports and `flags="S"` (SYN) — the half-open scan.
6. **Keyword arg** — `timeout=2` stops waiting after 2 seconds so unanswered ports don't hang the scan.

⇄ DISTINGUISH

"Custom packet crafting / raw packets" → Scapy. The two dead giveaways are the `/` layer-stacking and `sr1()`. Contrast with Impacket, which works at the *protocol/auth* level, not the raw-packet level.

EXAM FOCUS

- Builds packets field-by-field; `/` stacks layers (IP()/TCP()).
- ARP spoofing, VLAN hopping, port scan, DNS spoof, fuzzing (fuzz()).
- `sr1()` = send + receive one response.

`packet crafting` `raw sockets` `ARP spoofing` `VLAN hopping` `protocol fuzzing` `layer stacking` `sr1` `firewall evasion`

SEC 04 Breach & Attack Simulation

BAS platforms automate attacker behavior to **validate whether defenses actually detect and stop it** — continuously, not once a year. Three tools, and the exam loves confusing them. The whole section comes down to one three-way split.

4.1 Caldera

An open-source **adversary-emulation platform built by MITRE** — so it maps directly to **MITRE ATT&CK**. Server-agent model: the server commands, lightweight agents (Sandcat, Manx, Ragdoll) execute on targets. An **ability** = one ATT&CK technique; an **adversary** = a chain of abilities simulating a full threat actor. Its standout is **adaptive attack chaining** — parsers extract facts (creds, hostnames) from output and feed the next step, like a real attacker.

> ⚑ EXAM TELL
>
> Caldera = **MITRE**, ATT&CK-mapped, **full multi-stage operation chains**, adaptive. "Simulate a complete threat actor / end-to-end campaign" → Caldera.

EXAM FOCUS

- Built by MITRE; abilities = techniques, adversaries = chains.
- Server-agent architecture; adaptive chaining via extracted facts.
- Validates whether defenses detect specific ATT&CK techniques.

`MITRE ATT&CK` `adversary emulation` `abilities` `adversaries` `agents` `attack chain automation` `coverage gaps`

4.2 Infection Monkey

An open-source BAS tool (Guardicore/Akamai) that simulates **malware/worm propagation** — how far an attacker could spread *after* initial access. Architecture: **Monkey Island** (C2 server + attack map) and **Monkey Agent** (the propagating component). It spreads via known-vuln exploitation, credential reuse, default creds, and services (SMB/SSH/RDP/WinRM). It's **safe by design** — no encryption or data loss, self-terminates.

> ⇄ DISTINGUISH
>
> Infection Monkey = propagation. "How far could a breach spread?" / "test network segmentation / credential reuse" → Infection Monkey. Watch the naming trap: **Monkey Island belongs to Infection Monkey**, not Caldera.

EXAM FOCUS

- Simulates worm propagation post-compromise.
- Monkey Island (server) + Monkey Agent (spreader).
- Tests segmentation, credential reuse, lateral paths, patching.

`worm propagation` `network segmentation` `credential reuse` `Monkey Island` `lateral movement` `Zero Trust assessment`

4.3 Atomic Red Team

A library of small, self-contained tests (Red Canary), each simulating **one ATT&CK technique** in isolation. Tests are YAML with a technique ID (e.g. `T1003.001`), **attack commands AND cleanup commands**, run via `Invoke-AtomicTest`. Purpose: pinpoint **detection gaps** — run a test, check if the SIEM/EDR alerted; no alert = a coverage gap. Core of purple-team work.

POWERSHELL · ATOMIC RED TEAM

```powershell
Invoke-AtomicTest T1003.001 -ShowDetails   # list what the test does
Invoke-AtomicTest T1003.001 -CheckPrereqs   # verify prerequisites
Invoke-AtomicTest T1003.001                 # run the single technique
Invoke-AtomicTest T1003.001 -Cleanup        # restore pre-test state
```

⚑ EXAM TELL

Atomic Red Team = **individual technique** testing via `Invoke-AtomicTest`, maintained by **Red Canary (not MITRE)**. "Does our tool detect *this one* technique?" → Atomic Red Team.

EXAM FOCUS

- Tests one technique at a time; run with Invoke-AtomicTest.
- Each test includes attack + cleanup commands (YAML).
- Purpose = detection-gap identification; by Red Canary.

`atomic tests` `Invoke-AtomicTest` `detection gaps` `cleanup commands` `YAML` `Red Canary` `purple team`

The three-way split (memorize this)

`Caldera = full chains (MITRE)` `Infection Monkey = propagation` `Atomic Red Team = single techniques`

One technique detection test → **Atomic**. Full campaign / threat-actor TTPs → **Caldera**. How far a breach spreads / segmentation → **Infection Monkey**.

LAB Script Lab — Spot & Fix

The two skills the exam actually tests: **spotting** what a snippet does, and **fixing** a script that's subtly wrong. Try each before revealing — the answer key explains the reasoning, not just the fix. These bugs are the exact ones the exam plants.

Part 1 · Spot the script

Identify the language, what it does, and the tell.

SPOT 1 **What does this produce?**

SNIPPET

```
sort users.txt | uniq -c | sort -rn
```

ANSWER KEY

`Bash · frequency count`

Counts how many times each unique line appears, then lists them most-frequent first. **Read it:** sort (so duplicates are adjacent) → `uniq -c` (collapse + count) → `sort -rn` (reverse numeric sort on the counts).

Tell: `uniq` only works on sorted input, so `sort | uniq -c | sort -rn` is the canonical "find the most common value" idiom — great for spotting reused passwords or common open ports.

SPOT 2 **Which tool, and what attack?**

SNIPPET

```
GetUserSPNs.py corp.local/jdoe:'Winter2025' -dc-ip 10.0.0.10 -request
```

ANSWER KEY

Python · Impacket · Kerberoasting

An Impacket tool (`.py`) requesting Kerberos service tickets for accounts that have SPNs, so their hashes can be cracked offline. The `domain/user:pass@`-style credential and `-dc-ip` are Impacket fingerprints.

Tell: SPNs → Kerberoasting → GetUserSPNs.py. If the request targeted pre-auth-disabled accounts instead, it'd be `GetNPUsers.py` (AS-REP roasting).

SPOT 3 **Language and purpose?**

SNIPPET

```
send(Ether()/Dot1Q(vlan=1)/Dot1Q(vlan=20)/IP(dst="10.20.0.1")/ICMP())
```

ANSWER KEY

Python · Scapy · VLAN hopping

A double-tagged 802.1Q frame — two stacked `Dot1Q` layers — is the **VLAN hopping (double-tagging)** technique: the first tag is stripped by the switch, the inner tag lands the packet on VLAN 20.

Tell: The `/` layer-stacking is pure Scapy. Two `Dot1Q` layers = double-tagging = VLAN hopping.

Part 2 · Fix the script

Each runs without crashing but does the wrong thing. Find the bug, then reveal the fix.

FIX 1 **Only the last host's results survive**

BUGGY · BASH

```
for h in $(cat hosts.txt); do
    nmap -sV "$h" > scan.txt
done
```

× Every loop overwrites the same file

THE FIX

FIXED · BASH

```
for h in $(cat hosts.txt); do
    nmap -sV "$h" > "scan_${h}.txt"    # per-host file
done
# or, to collect into one file, use append:  nmap -sV "$h" >> scan.txt
```

✓ Per-host filenames — or >> to append

Why: `>` truncates the file on every iteration, so only the final host remains. Use a unique filename per host, or switch to `>>` to accumulate. This `>`-vs-`>>` distinction is a favorite exam trap.

FIX 2 — Errors still spam the screen

BUGGY · BASH

```
nmap -sV 10.0.0.0/24 2>&1 > all_output.txt
```

× Redirect order sends stderr to the terminal

THE FIX

FIXED · BASH

```
nmap -sV 10.0.0.0/24 > all_output.txt 2>&1
```

✓ Put 2>&1 AFTER the file redirect

Why: `2>&1` means "point stderr wherever stdout points *right now*." In the buggy line stdout still points at the terminal when that runs, so errors go to screen and only stdout reaches the file. Redirect stdout to the file first, *then* tie stderr to it.

FIX 3 — cut returns the whole line, not the username

BUGGY · BASH

```
cut -f1 /etc/passwd | sort
```

× Default delimiter is TAB, but passwd uses colons

THE FIX

FIXED · BASH

```
cut -d: -f1 /etc/passwd | sort
```

✓ Set the delimiter with -d:

Why: `cut` defaults to a tab delimiter. Since `/etc/passwd` has no tabs, field 1 is the entire line. `-d:` tells cut the fields are colon-separated, so field 1 is the username.

FIX 4 — PowerShell filter throws a parser error

BUGGY · POWERSHELL

```
Get-ADUser -Filter {PasswordNeverExpires = $true}
```

× = is assignment; filters need -eq

THE FIX

FIXED · POWERSHELL

```
Get-ADUser -Filter {PasswordNeverExpires -eq $true} -Properties *
```

✓ Use the -eq comparison operator

Why: PowerShell uses word operators — `-eq`, `-ne`, `-lt`, `-gt` — not `=` / `==`. A bare `=` is assignment and is invalid inside a filter. This operator style is itself a language tell.

FIX 5 **Nested Domain Admins are missing**

BUGGY · POWERSHELL

```powershell
Get-ADGroupMember -Identity "Domain Admins"
```

× Misses users who inherit membership via nested groups

THE FIX

FIXED · POWERSHELL

```powershell
Get-ADGroupMember -Identity "Domain Admins" -Recursive
```

✓ Add -Recursive to expand nested groups

Why: Without `-Recursive`, only direct members are returned. If a group is a member of Domain Admins, its users are effectively admins but stay hidden — exactly the accounts you're hunting.

FIX 6 **Scapy scan errors on the response**

BUGGY · PYTHON (SCAPY)

```python
syn = TCP(dport=80, flags="S") / IP(dst="192.168.1.1")
resp = sr(syn, timeout=2)
if resp[TCP].flags == 0x02:
    print("open")
```

× Layers reversed · sr returns a tuple · wrong flag value

THE FIX

FIXED · PYTHON (SCAPY)

```python
syn = IP(dst="192.168.1.1") / TCP(dport=80, flags="S")   # IP outer, TCP inner
resp = sr1(syn, timeout=2)                               # sr1 = single packet
if resp and resp[TCP].flags == 0x12:                     # 0x12 = SYN-ACK
    print("open")
```

✓ IP()/TCP() order · sr1() · 0x12 (SYN-ACK)

Three bugs: (1) stacking is outer→inner, so `IP()/TCP()`, not the reverse. (2) `sr()` returns a *(answered, unanswered)* tuple — indexing `[TCP]` fails; use `sr1()` for a single response. (3) An open port replies **SYN-ACK = 0x12**; `0x02` is a plain SYN. Also guard for `None` on no reply.

MAP Master Correlation

The exam's favorite move: a scenario, and "which tool?" Match the **target** (AD, SQL, packets, propagation) and the **phase** (enumerate / exploit / simulate) to the tool.

PowerShell — by target

SCENARIO	TOOL
Find where Domain Admins are logged in	PowerView
Escalate privileges on a local Windows host	PowerSploit (PowerUp)
Dump credentials from memory	PowerSploit (Invoke-Mimikatz)
Attack SQL Server / find linked servers	PowerUpSQL
Enumerate accounts with SPNs (stealthy)	AD Search (native cmdlets)
Visualize AD attack paths graphically	BloodHound

Python — Impacket vs. Scapy

SCENARIO	TOOL
Windows auth / SMB / Kerberos / NTLM	Impacket
NTLM relay attack	Impacket (ntlmrelayx.py)
Credential dumping from AD	Impacket (secretsdump.py)
Kerberoasting	Impacket (GetUserSPNs.py)
Raw packet crafting / custom scan	Scapy
ARP spoofing · VLAN hopping · fuzzing	Scapy

BAS — the three-way differentiator

SCENARIO	TOOL
Test if one ATT&CK technique triggers an alert	Atomic Red Team
Simulate a complete multi-stage campaign	Caldera
Test how far malware could spread	Infection Monkey
Validate network segmentation / credential reuse	Infection Monkey
Simulate a specific threat actor's TTPs	Caldera
Test individual detection rules	Atomic Red Team

KEYS Keyword Master List

PowerShell

post-exploitation
living off the land
in-memory PowerUp
unquoted service paths
Domain Admins
session discovery
SQL Server SPNs
xp_cmdshell
linked servers
native cmdlets
Kerberoasting

Bash

stdin/stdout/stderr
pipe redirect append
grep awk sed cut
sort uniq /dev/null
tee
positional parameters

Python

Impacket Scapy
NTLM relay
Kerberoasting
secretsdump psexec
ntlmrelayx
packet crafting
layer stacking
ARP spoofing
protocol fuzzing
raw sockets

BAS

MITRE ATT&CK
adversary emulation
abilities agents
Monkey Island
atomic tests
Invoke-AtomicTest
detection gaps
network segmentation
propagation purple team

EXAM Final Strategy for Domain 4.10

01 **Tool identification = target + phase.** Windows AD → PowerView/AD Search; SQL Server → PowerUpSQL; local privesc → PowerSploit/PowerUp; Windows protocols from Linux → Impacket; raw packets → Scapy.

02 **Script reading = trace left to right.** Identify the language first, then walk each pipe/redirect and say what it does to the data stream. Narrate it as one sentence.

03 **BAS three-way split.** Caldera (full chains, MITRE) · Infection Monkey (propagation, Monkey Island) · Atomic Red Team (single techniques, Invoke-AtomicTest, Red Canary).

04 **Know the planted bugs.** `>` vs `>>`, `2>&1` ordering, `uniq` needs `sort`, `cut -d`, `-eq` not `=`, `-Recursive`, `IP()/TCP()` order, `sr1()`, SYN-ACK = `0x12`.

05 **Impacket tools by job.** secretsdump (dump) · psexec/wmiexec (exec) · GetUserSPNs (Kerberoast) · GetNPUsers (AS-REP) · ntlmrelayx (relay) · ticketer (Golden/Silver).

Study aid — condensed for review. Verify tool syntax and current exam objectives against official CompTIA materials.